INSTITUTO TECNOLÓGICO SUPERIOR SUDAMERICANO

Carrera de Tecnología en Desarrollo de Software

Informe técnico:
API REST de usuarios temporales con NestJS

Dato	Detalle
Asignatura	Herramientas informáticas para el despliegue de diagramas
Código	TG-DS-PAIT-03-3
Ciclo	Tercero
Periodo académico	Septiembre 2026 – febrero 2027
Estudiante	Boris Xavier Viracocha Barros

Cuenca, Ecuador
Octubre de 2026

Resumen

El presente informe documenta el diseño, la implementación y la verificación de una API REST de usuarios temporales desarrollada con NestJS. Los usuarios, caracterizados por un identificador, un nombre, un correo electrónico y una edad, se almacenan únicamente en la memoria del servidor, sin utilizar Prisma ni una base de datos. La aplicación se organizó en un módulo, un controlador y un servicio, y expone operaciones de consulta, creación y eliminación mediante los métodos HTTP GET, POST y DELETE, además de los métodos HEAD y OPTIONS. También se incorporó un comando de terminal que crea usuarios compartiendo el mismo almacenamiento que la API, para lo cual se resolvió el problema de que cada proceso de Node.js dispone de su propia memoria. Las pruebas se realizaron con Postman, y se documentan tanto los resultados correctos como los errores detectados y su corrección. Finalmente, se declara el uso de inteligencia artificial durante el proceso y se analizan las limitaciones de la solución.

Palabras clave: NestJS, API REST, métodos HTTP, HEAD, OPTIONS, Postman, almacenamiento en memoria.

1. Introducción

Las aplicaciones backend procesan solicitudes, gestionan información y generan respuestas para los clientes que consumen sus servicios. Esa comunicación se apoya en el protocolo HTTP, que define los métodos, la estructura de las solicitudes y respuestas y los códigos de estado (Fielding y Reschke, 2014; Mozilla Developer Network [MDN], 2024). Comprender estos elementos y aplicarlos en un servicio concreto es uno de los propósitos de la primera unidad de la asignatura.

En este trabajo se desarrolla una API de usuarios temporales con NestJS, un marco de trabajo para Node.js que organiza las aplicaciones en módulos, controladores y proveedores (NestJS, s. f.-a). La solución se limita deliberadamente a lo esencial: no utiliza Prisma ni una base de datos, y los datos se mantienen en la memoria del servidor mientras este permanece en ejecución.

1.1. Objetivo general

Desarrollar una API REST de usuarios temporales con NestJS, almacenados en memoria, que pueda ser consumida desde Postman y desde la terminal.

1.2. Objetivos específicos

- Implementar la consulta, la creación y la eliminación de usuarios mediante los métodos GET, POST y DELETE.
- Aplicar los métodos HEAD y OPTIONS, e identificar su diferencia con GET y su utilidad para informar los métodos permitidos.
- Organizar la aplicación en módulo, controlador y servicio para separar responsabilidades.
- Permitir la creación de usuarios desde la terminal compartiendo los mismos datos que la API.
- Verificar el funcionamiento de la API mediante pruebas en Postman.

2. Análisis del problema y requerimientos

2.1. Descripción del problema

Se requiere un servicio que gestione usuarios de forma temporal. Cada usuario tiene los atributos `id`, `nombre`, `email` y `edad`. Los usuarios pueden agregarse, consultarse y eliminarse, y deben poder crearse tanto desde un cliente HTTP (Postman) como desde la terminal.

2.2. Requerimientos funcionales

Tabla 1. *Requerimientos funcionales*

Código	Requerimiento
RF1	Crear un usuario con nombre, correo electrónico y edad, asignándole un identificador único.

Código	Requerimiento
RF2	Obtener la lista de todos los usuarios.
RF3	Obtener un usuario a partir de su identificador.
RF4	Eliminar un usuario a partir de su identificador.
RF5	Atender solicitudes HEAD sobre la colección y sobre un usuario.
RF6	Atender solicitudes OPTIONS indicando los métodos HTTP permitidos.
RF7	Crear usuarios desde la terminal mediante el comando `npm run usuario`.

2.3. Restricciones

- No se utiliza Prisma ni ninguna base de datos.
- Los usuarios se almacenan temporalmente en memoria.
- No se incorporan tecnologías que no sean necesarias para el alcance definido.
- El código se desarrolla en Visual Studio Code y la API se prueba con Postman.

3. Marco teórico

3.1. Protocolo HTTP y métodos

HTTP es un protocolo de comunicación entre clientes y servidores en el que el cliente envía solicitudes y el servidor devuelve respuestas (MDN, 2024). La especificación de semántica de HTTP/1.1 define los métodos utilizados en este trabajo (Fielding y Reschke, 2014):

- **GET:** solicita la representación de un recurso.
- **POST:** envía datos para que el servidor los procese; en este trabajo se utiliza para crear un usuario.
- **DELETE:** solicita la eliminación de un recurso.
- **HEAD:** es idéntico a GET, salvo que el servidor no debe enviar cuerpo en la respuesta; los encabezados deberían ser los mismos que los de la respuesta a un GET.
- **OPTIONS:** solicita información sobre las opciones de comunicación disponibles para el recurso. Los métodos admitidos se comunican mediante el encabezado `Allow`.

Los códigos de estado empleados son 200 (OK), 201 (Created), 204 (No Content), 400 (Bad Request) y 404 (Not Found), todos definidos en la misma especificación (Fielding y Reschke, 2014).

3.2. API REST

REST es un estilo arquitectónico para sistemas distribuidos en la web, descrito por Fielding (2000). En una API de este tipo, los recursos se identifican mediante direcciones y se manipulan con los métodos HTTP (Richardson et al., 2013). En este trabajo, el recurso es `/users`.

3.3. NestJS: módulos, controladores y proveedores

En NestJS, los controladores atienden las solicitudes HTTP y delegan las tareas más complejas en proveedores, que son clases declaradas en un módulo (NestJS, s. f.-b). Los módulos agrupan los controladores y los proveedores que forman una parte de la aplicación (NestJS, s. f.-a). Esta organización favorece la separación de responsabilidades y se aplica directamente en la solución desarrollada.

4. Metodología y herramientas

El trabajo se desarrolló de forma incremental, por etapas: creación del proyecto, revisión de la estructura, implementación de usuarios, pruebas con Postman, implementación de HEAD, implementación de OPTIONS, creación desde la terminal, pruebas integrales y explicación del funcionamiento. Cada etapa se realizó y comprobó antes de pasar a la siguiente.

Tabla 2. *Herramientas utilizadas*

Herramienta	Uso en el proyecto
Node.js y npm	Entorno de ejecución y gestor de paquetes y scripts.
NestJS CLI	Creación del proyecto y generación de módulo, controlador y servicio.
Visual Studio Code	Edición del código y terminal integrada.
Postman	Envío de solicitudes HTTP y verificación de las respuestas.
Asistente de IA (Claude)	Apoyo en la guía, el código y el diagnóstico de errores (ver sección 9).

5. Diseño e implementación

5.1. Creación y ejecución del proyecto

El proyecto se creó desde la terminal, se abrió en Visual Studio Code y se ejecutó en modo desarrollo:

```
nest new usuarios-api
cd usuarios-api
code .
npm run start:dev
```

Al elegir `npm` como gestor de paquetes, NestJS genera la estructura base. Al abrir `http://localhost:3000` en el navegador se muestra el mensaje `Hello World!`, lo que confirma que el servidor funciona. El modo `start:dev` reinicia el servidor cada vez que se guarda un archivo.

5.2. Estructura del proyecto

De la estructura generada se utilizaron los siguientes elementos; el resto (por ejemplo, `app.controller.ts` y `app.service.ts`) se mantuvo sin cambios.

```
usuarios-api/
|-- src/
|   |-- main.ts            (arranca el servidor en el puerto 3000)
|   |-- app.module.ts      (módulo principal)
|   `-- users/
|       |-- users.module.ts
|       |-- users.controller.ts
|       `-- users.service.ts
|-- scripts/
|   `-- crear-usuario.js   (creación desde la terminal)
`-- package.json           (script "usuario")
```

Los archivos del módulo se generaron con los siguientes comandos de NestJS CLI, donde la opción `--no-spec` evita crear archivos de prueba que no se utilizan:

```
nest g module users
nest g controller users --no-spec
nest g service users --no-spec
```

5.3. Responsabilidad de cada componente

- **UsersModule:** agrupa el controlador y el servicio de usuarios y se importa en el módulo principal.
- **UsersController:** recibe las solicitudes HTTP, valida los datos de entrada y llama al servicio. No guarda datos.
- **UsersService:** contiene la lógica de usuarios y administra el arreglo en memoria.

5.4. Módulo

Código 1. *src/users/users.module.ts*

```
import { Module } from '@nestjs/common';
import { UsersController } from './users.controller';
import { UsersService } from './users.service';

@Module({
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```

5.5. Servicio

El servicio mantiene un arreglo `usuarios` y un contador `siguienteId`. Si un usuario no existe, lanza `NotFoundException`, que NestJS traduce en una respuesta 404.

Código 2. *src/users/users.service.ts*

```
import { Injectable, NotFoundException } from '@nestjs/common';

export interface Usuario {
  id: number;
  nombre: string;
  email: string;
  edad: number;
```

```
}

@Injectable()
export class UsersService {
  private usuarios: Usuario[] = [];
  private siguienteId = 1;

  crear(nombre: string, email: string, edad: number): Usuario {
    const usuario: Usuario = { id: this.siguienteId++, nombre, email, edad };
    this.usuarios.push(usuario);
    return usuario;
  }

  obtenerTodos(): Usuario[] {
    return this.usuarios;
  }

  obtenerUno(id: number): Usuario {
    const usuario = this.usuarios.find((u) => u.id === id);
    if (!usuario) {
      throw new NotFoundException(`Usuario con id ${id} no encontrado`);
    }
    return usuario;
  }

  eliminar(id: number): { mensaje: string } {
    const indice = this.usuarios.findIndex((u) => u.id === id);
    if (indice === -1) {
      throw new NotFoundException(`Usuario con id ${id} no encontrado`);
    }
    this.usuarios.splice(indice, 1);
    return { mensaje: `Usuario ${id} eliminado` };
  }
}
```

5.6. Controlador

El controlador define las rutas bajo `/users`. El parámetro `:id` se convierte a número con `ParseIntPipe`, que responde con un error 400 si el valor no es numérico. En el método de creación se valida que existan nombre, correo y una edad numérica; de lo contrario, se responde con 400.

Código 3. *src/users/users.controller.ts*

```
import {
  BadRequestException,
  Body,
  Controller,
  Delete,
  Get,
  Head,
  Header,
  HttpCode,
  Options,
  Param,
  ParseIntPipe,
  Post,
  Res,
} from '@nestjs/common';
import type { Response } from 'express';
import { UsersService } from './users.service';

export interface CrearUsuarioBody {
```

```
  nombre: string;
  email: string;
  edad: number;
}

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Head()
  contarUsuarios(@Res({ passthrough: true }) res: Response) {
    const total = this.usersService.obtenerTodos().length;
    res.setHeader('X-Total-Count', String(total));
  }

  @Head(':id')
  verificarUsuario(@Param('id', ParseIntPipe) id: number) {
    this.usersService.obtenerUno(id);
  }

  @Options()
  @Header('Allow', 'GET, HEAD, POST, OPTIONS')
  @HttpCode(204)
  opcionesUsuarios() {}

  @Options(':id')
  @Header('Allow', 'GET, HEAD, DELETE, OPTIONS')
  @HttpCode(204)
  opcionesUsuario() {}

  @Get()
  obtenerTodos() {
    return this.usersService.obtenerTodos();
  }

  @Get(':id')
  obtenerUno(@Param('id', ParseIntPipe) id: number) {
    return this.usersService.obtenerUno(id);
  }

  @Post()
  crear(@Body() body: CrearUsuarioBody) {
    if (!body.nombre || !body.email || typeof body.edad !== 'number') {
      throw new BadRequestException(
        'Se requieren: nombre (texto), email (texto) y edad (número)',
      );
    }
    return this.usersService.crear(body.nombre, body.email, body.edad);
  }

  @Delete(':id')
  eliminar(@Param('id', ParseIntPipe) id: number) {
    return this.usersService.eliminar(id);
  }
}
```

5.7. Endpoints implementados

Tabla 3. *Endpoints de la API*

Método	Ruta	Descripción	Estados
GET	`/users`	Devuelve todos los usuarios.	200

Método	Ruta	Descripción	Estados
GET	`/users/:id`	Devuelve un usuario.	200, 404
POST	`/users`	Crea un usuario a partir del JSON recibido.	201, 400
DELETE	`/users/:id`	Elimina un usuario.	200, 404
HEAD	`/users`	Devuelve solo encabezados, con `X-Total-Count`.	200
HEAD	`/users/:id`	Verifica si el usuario existe.	200, 404
OPTIONS	`/users`	Informa los métodos con `Allow`.	204
OPTIONS	`/users/:id`	Informa los métodos con `Allow`.	204

5.8. Implementación de HEAD y OPTIONS

Para **HEAD**, el controlador no devuelve cuerpo y agrega el encabezado personalizado `X-Total-Count` con la cantidad de usuarios. De esta forma, el cliente obtiene un dato útil sin descargar la lista. La diferencia con GET es que este último devuelve el contenido de los usuarios en el cuerpo, mientras que HEAD solo entrega los encabezados (Fielding y Reschke, 2014).

Para **OPTIONS**, el controlador responde con el código 204 y fija el encabezado `Allow` mediante `@Header`. En la colección se anuncian los métodos GET, HEAD, POST y OPTIONS; para un usuario individual, GET, HEAD, DELETE y OPTIONS.

5.9. Creación de usuarios desde la terminal

Problema. Si el comando de terminal importara el servicio y creara el usuario por su cuenta, se ejecutaría en un proceso de Node.js distinto al del servidor. Cada proceso tiene su propia memoria y, por tanto, su propio arreglo `usuarios`: el usuario se crearía en el proceso del comando y desaparecería al terminar este, sin que el servidor consultado desde Postman llegara a conocerlo.

Solución. El comando no guarda datos: envía una solicitud POST a `/users` del servidor en ejecución, igual que Postman. Así, ambos clientes utilizan el mismo servicio y el mismo arreglo, sin base de datos ni archivos adicionales. El único requisito es que el servidor esté en ejecución en otra terminal.

Código 4. *scripts/crear-usuario.js*

```
const [nombre, email, edad] = process.argv.slice(2);

if (!nombre || !email || !edad) {
  console.log('Uso: npm run usuario -- "Nombre" "correo@email.com" edad');
  process.exit(1);
}

async function crearUsuario() {
  try {
    const respuesta = await fetch('http://localhost:3000/users', {
```

```
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ nombre, email, edad: Number(edad) }),
    });

    const datos = await respuesta.json();

    if (!respuesta.ok) {
      console.log('Error:', datos);
      process.exit(1);
    }

    console.log('Usuario creado:', datos);
  } catch (error) {
    console.log('No se pudo conectar. ¿Está corriendo npm run start:dev?');
    process.exit(1);
  }
}

crearUsuario();
```

El comando se registra en la sección `scripts` de `package.json` y se utiliza de la siguiente manera:

```
"usuario": "node scripts/crear-usuario.js"

npm run usuario -- "Boris" "boris@email.com" 17
```

`process.argv.slice(2)` toma los argumentos escritos tras el comando, y `Number(edad)` convierte la edad a número, porque desde la terminal todos los argumentos llegan como texto.

6. Funcionamiento general del sistema

La Figura 1 resume cómo se relacionan los componentes. Tanto la terminal como Postman actúan como clientes HTTP del mismo servidor NestJS. El servidor dirige cada solicitud al controlador, este llama al servicio y el servicio opera sobre el arreglo en memoria. La respuesta recorre el camino inverso hasta el cliente que realizó la solicitud.

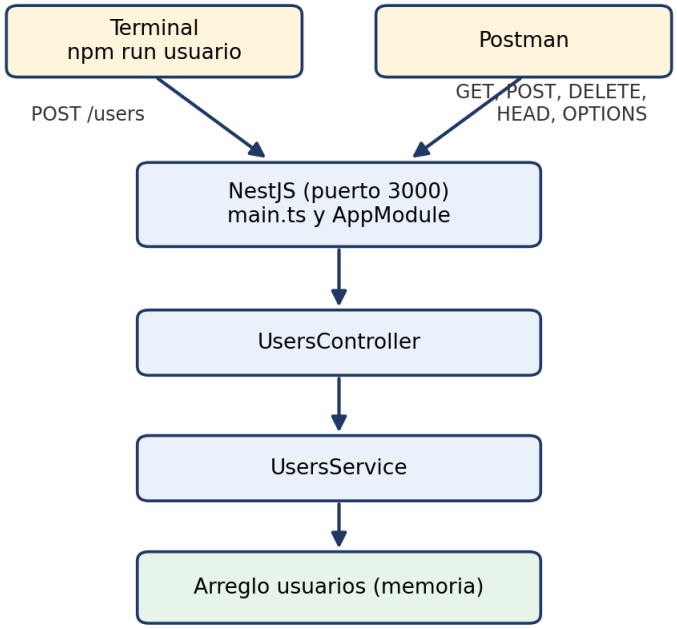


Figura 1. Flujo entre terminal, Postman, NestJS, controlador, servicio y usuarios. Elaboración propia.

7. Pruebas y resultados

7.1. Entorno de prueba

Las pruebas se realizaron con Postman sobre `http://localhost:3000`, con el servidor en ejecución mediante `npm run start:dev`. Para el POST se configuró el cuerpo como `raw` en formato JSON, con el encabezado `Content-Type: application/json`.

7.2. Creación de un usuario (POST)

Se envió el siguiente cuerpo a `POST /users`:

```
{
  "nombre": "Boris",
  "email": "boris@email.com",
  "edad": 17
}
```

El servidor respondió con el código 201 (Created) y el usuario creado con el identificador 1 (Figura 2), lo que cumple el requerimiento RF1.

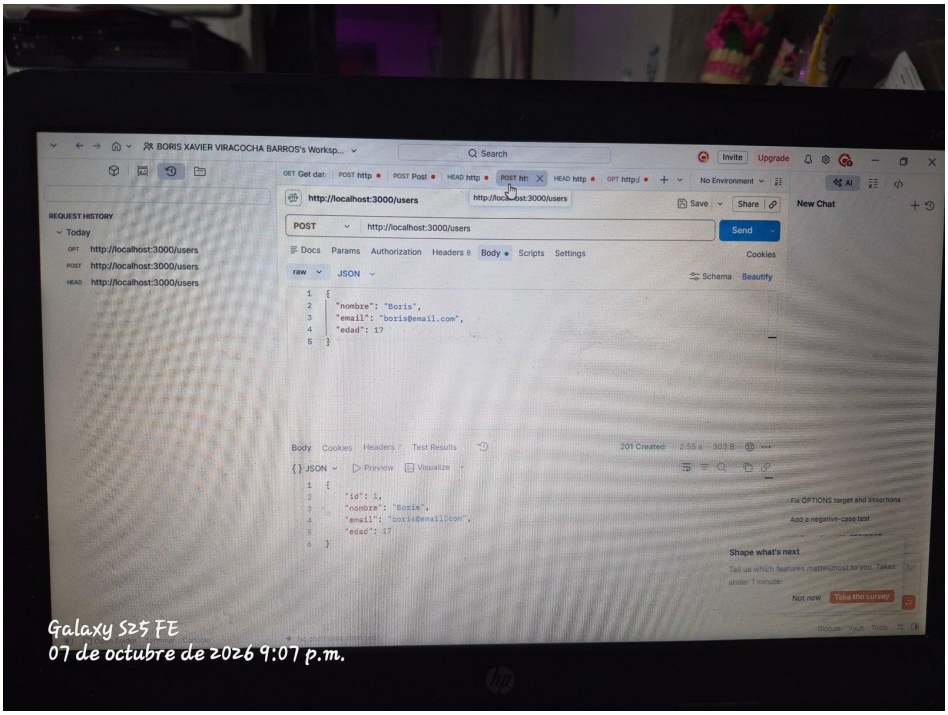


Figura 2. Solicitud POST a /users con respuesta 201 Created. Captura de Postman.

7.3. Errores detectados en HEAD y OPTIONS y su corrección

Durante las pruebas se detectaron dos problemas, que se documentan por ser parte del proceso de verificación.

HEAD sin el encabezado esperado. La solicitud `HEAD /users` respondió 200, pero sin el encabezado `X-Total-Count` (Figura 3). La longitud del contenido indicada era de 2, valor que corresponde a un arreglo vacío `[]`, lo que indica que la respuesta provenía del método GET y no del manejador HEAD.

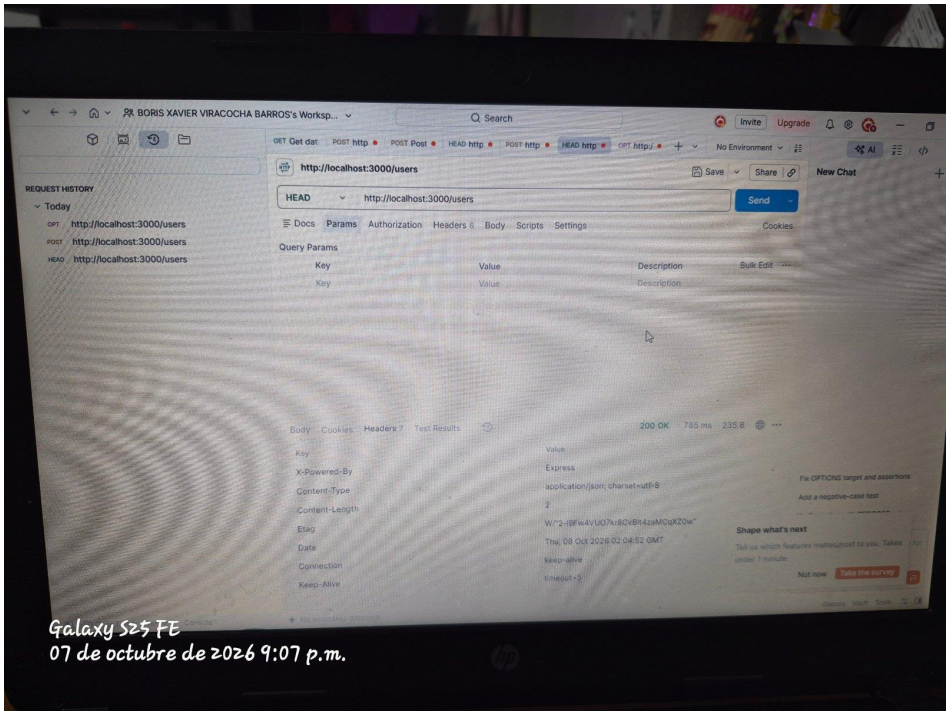


Figura 3. HEAD a /users sin el encabezado X-Total-Count. Captura de Postman.

OPTIONS no encontrado. La solicitud `OPTIONS /users` respondió 404 Not Found (Figura 4), es decir, el servidor no tenía una ruta OPTIONS registrada.

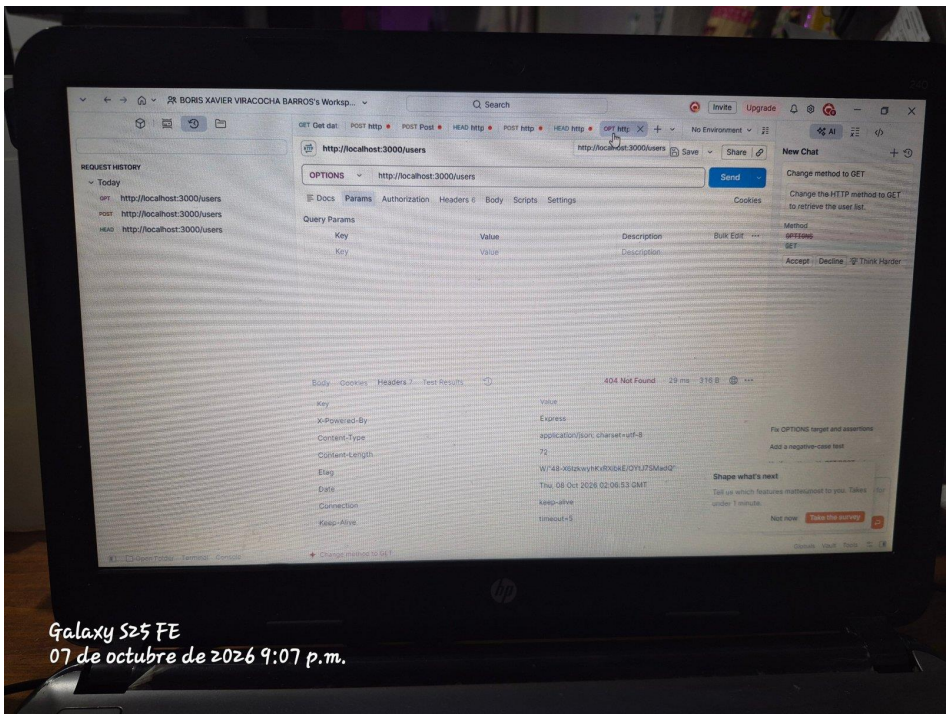


Figura 4. OPTIONS a /users con respuesta 404 Not Found. Captura de Postman.

Diagnóstico y corrección. Ambos síntomas indicaban que los métodos `@Head()` y `@Options()` no estaban activos en el controlador. Se reemplazó el archivo por la versión completa del Código 3, en la que los manejadores de HEAD y OPTIONS se declaran antes que

los de GET. El orden es relevante: de acuerdo con lo observado, si el manejador GET se declara primero, Express atiende con él las solicitudes HEAD y el manejador HEAD nunca se ejecuta.

7.4. Resultados tras la corrección

Con el controlador corregido se repitieron las pruebas (Figuras 5, 6 y 7).

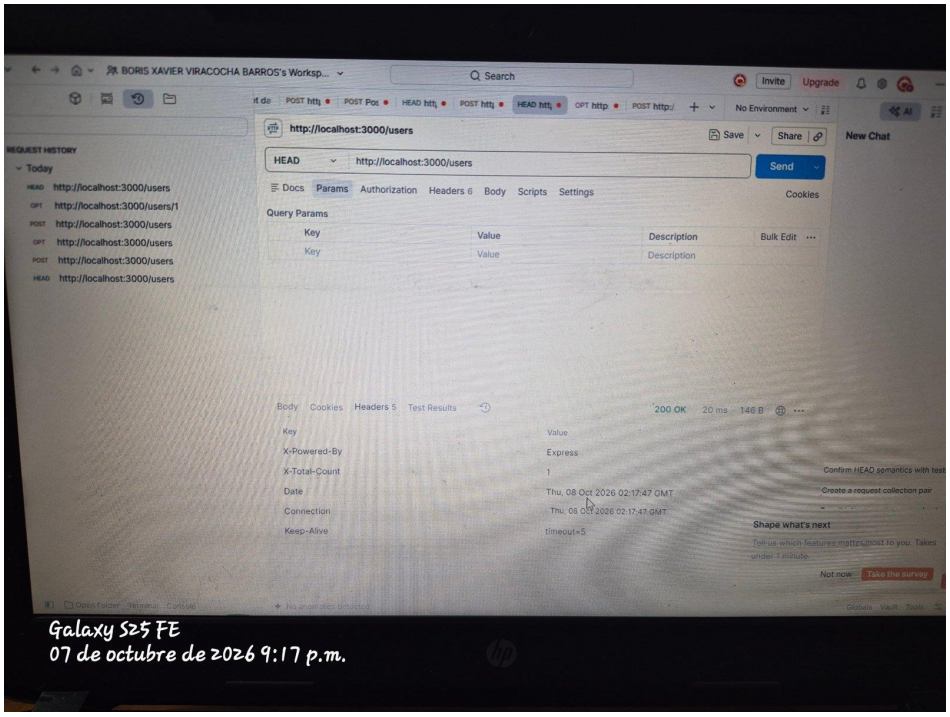


Figura 5. HEAD a /users con 200 OK y encabezado X-Total-Count: 1. Captura de Postman.

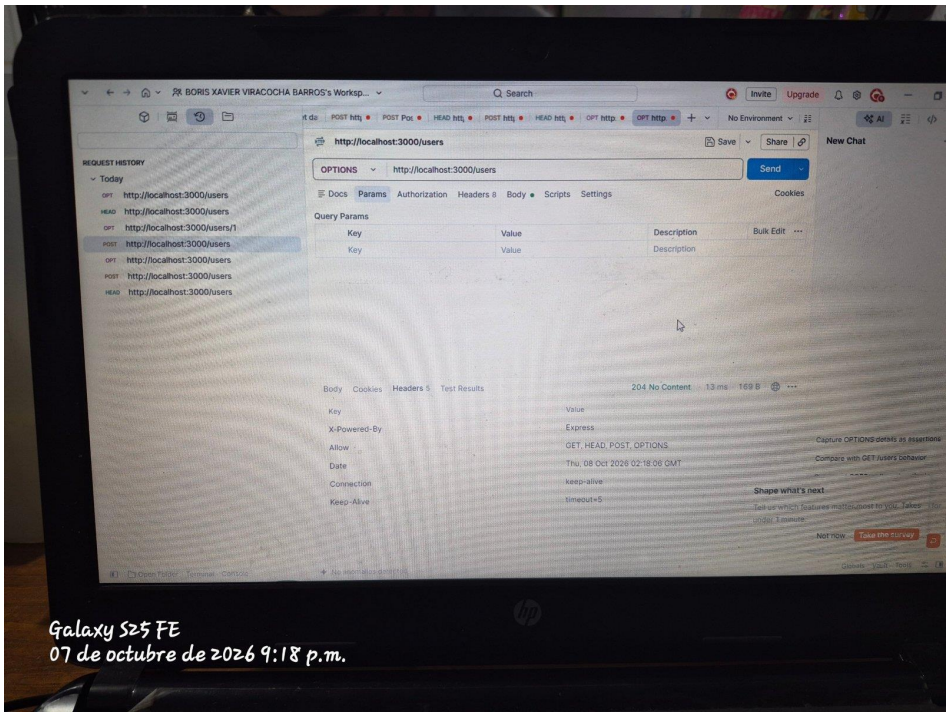


Figura 6. OPTIONS a /users con 204 No Content y Allow: GET, HEAD, POST, OPTIONS. Captura de Postman.

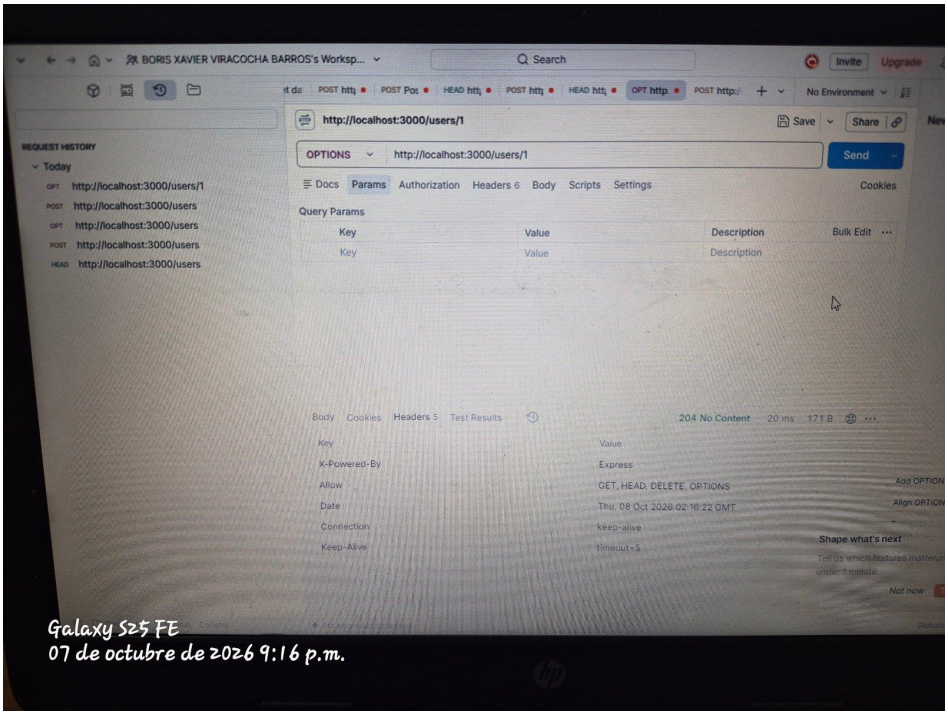


Figura 7. OPTIONS a /users/1 con 204 No Content y Allow: GET, HEAD, DELETE, OPTIONS. Captura de Postman.

Tabla 4. *Resumen de las pruebas con evidencia*

Solicitud	Resultado	Evidencia	Figura
POST `/users`	201 Created, `id: 1`	Usuario creado con los datos enviados.	2
HEAD `/users` (antes)	200, sin `X-Total-Count`	Error detectado.	3
OPTIONS `/users` (antes)	404 Not Found	Error detectado.	4
HEAD `/users` (después)	200, `X-Total-Count: 1`	Cuerpo vacío y encabezado presente.	5
OPTIONS `/users` (después)	204, `Allow` correcto	Métodos de la colección.	6
OPTIONS `/users/1`	204, `Allow` correcto	Métodos de un usuario.	7

7.5. Procedimiento de pruebas integrales definido

Para validar el conjunto de la solución, incluida la creación desde la terminal y la coherencia entre ambos clientes, se definió la siguiente secuencia, que parte con el servidor recién iniciado y la lista vacía:

Tabla 5. *Procedimiento de pruebas integrales*

N.°	Acción	Resultado esperado
1	POST `/users` desde Postman.	201 con `id: 1`.
2	GET `/users/1` desde Postman.	200 con el usuario.

N.º	Acción	Resultado esperado
3	`npm run usuario -- "Boris" "boris@email.com" 17`.	Mensaje `Usuario creado` con `id: 2`.
4	GET `/users` desde Postman.	Lista con los usuarios de Postman y de la terminal.
5	HEAD `/users`.	200, cuerpo vacío y `X-Total-Count: 2`.
6	OPTIONS `/users`.	204 y `Allow: GET, HEAD, POST, OPTIONS`.
7	DELETE `/users/1` y `/users/2`; luego GET `/users`.	200 en cada eliminación y lista vacía `[]`.

La prueba 4 es la más importante, pues demuestra que el usuario creado desde la terminal aparece al consultar desde Postman, es decir, que ambos clientes comparten el mismo almacenamiento.

8. Análisis crítico y limitaciones

- **Almacenamiento volátil.** Los usuarios existen solo mientras el servidor está en ejecución. Al detenerlo, y en modo `start:dev` al guardar cualquier archivo, el arreglo se vacía y los identificadores vuelven a comenzar en 1. Es coherente con el requisito de usuarios temporales, pero impide conservar la información.
- **Sin persistencia.** Al no utilizarse base de datos, no se garantiza integridad ni consistencia más allá de la ejecución del servidor.
- **Validación básica.** Se comprueba la presencia de nombre y correo y que la edad sea numérica, pero no se valida el formato del correo, el rango de la edad ni la duplicidad de correos.
- **Sin autenticación ni autorización.** Cualquier cliente que alcance el servidor puede crear y eliminar usuarios.
- **Sin pruebas automatizadas.** La verificación se realizó de forma manual con Postman.

Como mejoras futuras se plantean incorporar persistencia en una base de datos, ampliar las validaciones, añadir autenticación y escribir pruebas automatizadas.

9. Declaración de uso de inteligencia artificial

Herramienta utilizada: Claude (Anthropic), mediante una conversación en su interfaz de chat.

Alcance del uso:

- Orientación paso a paso para crear y ejecutar el proyecto NestJS y para usar Postman.
- Generación del código base del módulo, el controlador, el servicio y el script de terminal.
- Explicación conceptual de HEAD, OPTIONS y de la diferencia entre ambos y GET.

- Planteamiento de la solución al problema de memoria separada entre procesos.
- Diagnóstico de los errores de HEAD y OPTIONS a partir de las capturas de Postman enviadas durante la conversación.
- Elaboración del registro de trazabilidad de las consultas.

Participación y verificación del estudiante. El código se ejecutó en el equipo propio y los resultados se comprobaron en Postman. Los errores detectados se comunicaron mediante capturas de pantalla y, tras aplicar la corrección, las pruebas se repitieron hasta obtener las respuestas esperadas. Los conceptos sobre HEAD, OPTIONS y los códigos de estado se contrastan en este informe con la especificación HTTP/1.1 (Fielding y Reschke, 2014) y con la documentación de MDN (2024), y la organización en módulo, controlador y servicio con la documentación de NestJS (s. f.-a; s. f.-b).

El registro cronológico de las consultas realizadas, con su objetivo, la información obtenida y su relación con la consulta siguiente, se presenta en el documento adjunto *Registro de trazabilidad de la conversación*.

10. Conclusiones

- Se desarrolló una API REST de usuarios temporales con NestJS, organizada en módulo, controlador y servicio, que almacena los datos en memoria sin Prisma ni base de datos.
- Se implementaron los métodos GET, POST, DELETE, HEAD y OPTIONS, con los códigos de estado 200, 201, 204, 400 y 404 según cada caso.
- HEAD se diferencia de GET en que no devuelve cuerpo, lo que permite consultar información, como la cantidad de usuarios, sin descargar los datos. OPTIONS informa los métodos permitidos mediante el encabezado `Allow`.
- Las pruebas permitieron detectar y corregir dos errores relacionados con el orden de declaración de los manejadores HEAD y OPTIONS en el controlador.
- Al enviar el comando de terminal una solicitud POST al servidor, la terminal y Postman comparten los mismos usuarios, lo que resuelve el problema de la memoria separada entre procesos.
- La solución cumple el alcance definido, pero presenta limitaciones de persistencia, validación y seguridad que podrían abordarse en trabajos posteriores.

Referencias

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf

Fielding, R., y Reschke, J. (Eds.). (2014). *Hypertext Transfer Protocol (HTTP/1.1): Semantics and content* (RFC 7231). RFC Editor. https://www.rfc-editor.org/rfc/rfc7231

Mozilla Developer Network (MDN). (2024). *HTTP: The protocol every web developer must know*. https://developer.mozilla.org/en-US/docs/Web/HTTP

NestJS. (s. f.-a). *Modules*. https://docs.nestjs.com/modules

NestJS. (s. f.-b). *Providers*. https://docs.nestjs.com/providers

Richardson, L., Amundsen, M., y Ruby, S. (2013). *RESTful web APIs: Services for a changing world*. O'Reilly Media.